

Introdução ao R e ao RStudio

Mauricio Izumi (UFES)

31 de março de 2026

Instalando o R

1. Entre no site do R: <https://cran.r-project.org/>;
2. Clique em “Download R for Windows”;
3. Clique em “install R for the first time”;
4. Faça o download do arquivo (“Download R 4.3.3 for Windows”);
5. Para instalar, basta seguir as instruções do instalador.

Intalando o RStudio

1. Entre no site do RStudio: <https://posit.co/download/rstudio-desktop/>;
2. Clique em “DOWNLOAD RSTUDIO DESKTOP FOR WINDOWS” para iniciar o download, caso seu sistema operacional seja o Windows;
3. Para instalar, basta seguir as instruções do instalador.

Primeiros passos no R

Ao abrirmos o R, temos apenas uma **linha de comando** (Console). É nela que iremos digitar os nossos códigos. O sinal “>” (em vermelho) que inicia a linha indica que o console está pronto para receber os comandos. Quando aparecer um sinal de “+”, ao invés de “>”, significa que o comando que nós iniciamos não foi finalizado na linha anterior (isto é, tem algo faltando na sintaxe). Para cancelar e voltar ao “>”, pressione “Esc”.

Os comandos aparecem em vermelho e o resultado aparece em azul. Para executar os comandos pressione “Enter”.

Embora possamos digitar os nossos comandos diretamente na linha de comando do R, a prática mais comum é criar um **script**, um arquivo de texto, semelhante a um Bloco de Notas, em que iremos guardar todos os comandos que nós escrevemos. Para criar esse script clique em “Arquivo” e depois em “Novo script”. Aparecerá uma nova janela com um editor de texto. É nele que iremos escrever nossos códigos e salvá-los.

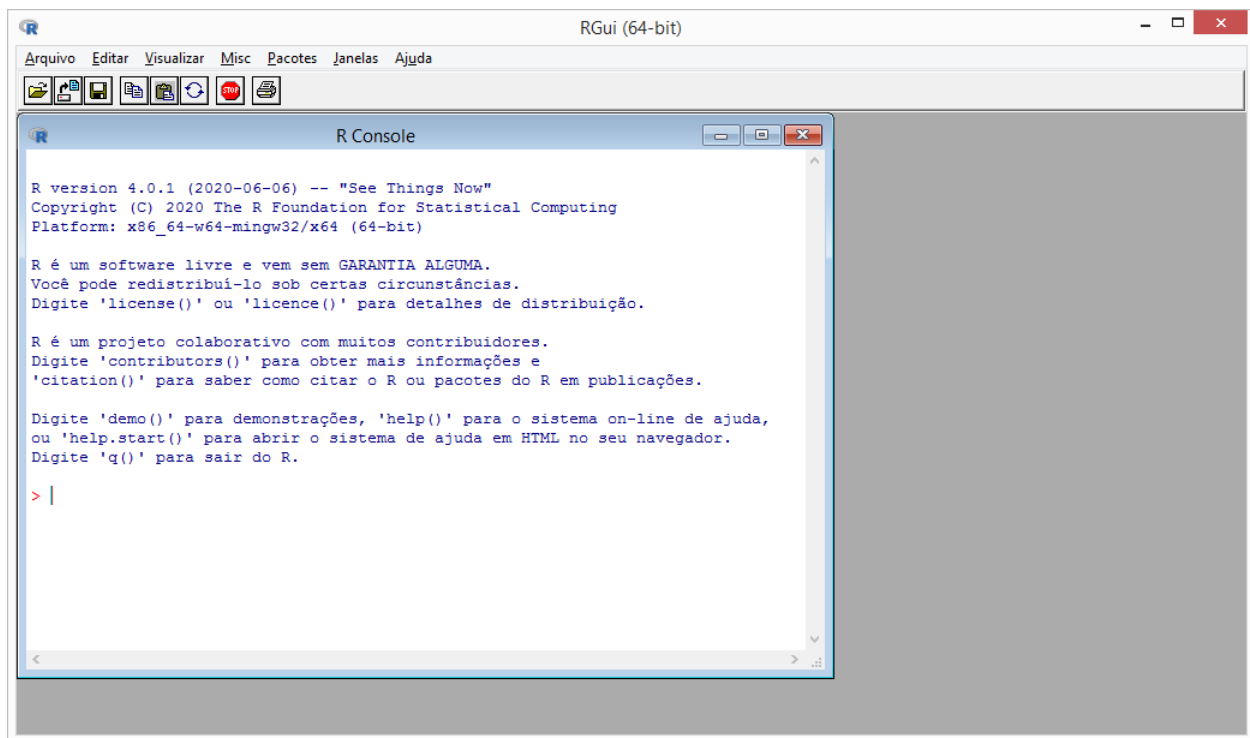
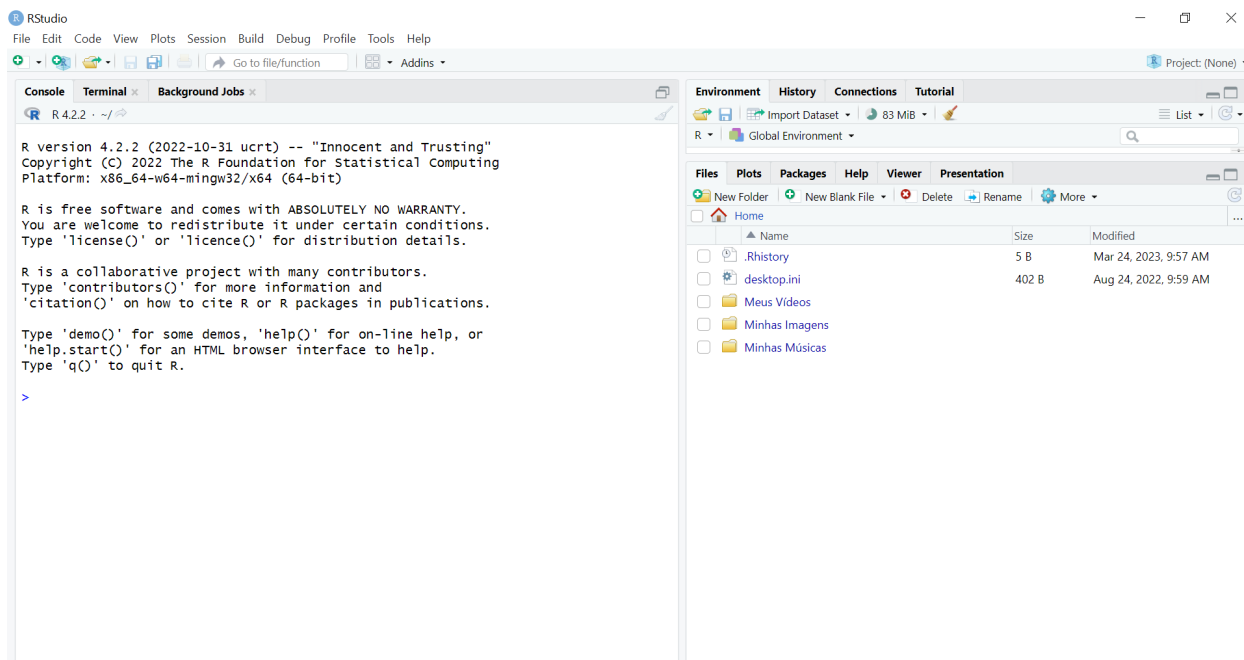


Figure 1: Este é o R

Para executar os códigos devemos selecionar as linhas de códigos de interesse e pressionar “Ctrl + R”. Esse comando executa o código que está no script em nosso console.

Primeiros passos no RStudio

A lógica é semelhante no **RStudio**.



Temos o console em que podemos digitar diretamente os nossos comandos, mas o mais comum é criar um **script**. Para criar esse script, basta clicar em “File” > “New File” > “R Script”.

Uma das poucas diferenças entre o R e o RStudio é o atalho para executar o código. Como vimos anteriormente, no R, utilizamos o “Ctrl + R”. No RStudio utilizamos o “Ctrl + Enter”.

Utilizando o R como uma calculadora

A forma mais simples de utilizar o R é usá-lo como uma calculadora. Podemos utilizar os principais operadores matemáticos:

- Adição: +
- Subtração: -
- Multiplicação: *
- Divisão: /
- Exponenciação: ^

Por exemplo, vamos calcular $2 + 2$:

```
2 + 2
```

```
## [1] 4
```

Veja que podemos utilizar parênteses nos cálculos para dar prioridade a algumas expressões, igual fazemos na matemática. Por exemplo:

```
2 + 2 * 3
```

```
## [1] 8
```

é diferente de

```
(2 + 2) * 3
```

```
## [1] 12
```

Funções

O R possui diversas **funções** que podemos utilizar para fazer cálculos e diversas outras coisas, como gráficos e manipulação de dados. Para utilizar uma função precisamos escrever o nome dela e colocar os seus **argumentos** entre parênteses. Vejamos mais algumas funções matemáticas para facilitar o entendimento. Para calcularmos $\log_2 8$:

```
log(8, base = 2)
```

```
## [1] 3
```

Portanto, a função “log” possui 2 argumentos, o primeiro é o número que iremos calcular o logaritmo (8). O segundo argumento indica a base em que iremos calcular o logaritmo (2).

Outras funções matemáticas importantes são:

- Raiz quadrada: `sqrt(x)`
- Valor absoluto: `abs(x)`
- Fatorial: `factorial(x)`

Pedindo ajuda

Muitas vezes não sabemos os argumentos de uma função ou não sabemos como utilizá-la. Para isso existe a função “help”:

```
help(log)
```

Por outro lado, se não soubermos nem mesmo o nome da função, podemos fazer uma busca no R de possíveis funções usando palavras-chave (em inglês). Por exemplo:

```
help.search("logarithms")
```

Em ambos os casos, o R abrirá algumas páginas com os resultados em seu navegador de internet.

No entanto, na prática, o mais comum é recorrermos ao **Google** para tirarmos as nossas dúvidas. Além disso, atualmente, há várias páginas, blogs e fóruns de discussão dedicados ao R (ou à programação como um todo). Por exemplo, veja o <https://stackoverflow.com/> e <https://www.r-bloggers.com/>.

Criando objetos

Vetores

O tipo de objeto mais simples do R é o **vetor**. Estes são os principais tipos de vetores que encontramos:

- Numéricos: `c(1,2,3,4)`
- Caracteres: `c("a","b","c","d")`
- Lógicos: `c(TRUE, FALSE, TRUE, FALSE)`

Em um vetor, todos os elementos devem ser do mesmo tipo. Não podemos ter, em um mesmo vetor, informações numéricas e de caracter, por exemplo.

Para criar um vetor seguimos a seguinte sintaxe:

```
x <- c(1,2,3,4)
x
```

```
## [1] 1 2 3 4
```

No exemplo acima, utilizamos a função “c” para concatenar os elementos 1, 2, 3 e 4 em um vetor. Ao utilizar o comando “<-” estamos atribuindo as informações desse vetor a um objeto de nome “x”. Vejam que a linha de código “x <- c(1,2,3,4)” não retorna nada. Isso ocorre porque estamos apenas atribuindo o vetor a “x”. Para observarmos o resultado, temos que executar “x”.

Podemos dar praticamente qualquer nome aos nossos objetos. Os nomes só não podem começar com um número (como “2x”) ou possuir espaços (como “x y”). Note também que o R é *case sensitive*, ou seja, faz diferença se utilizamos letras maiúsculas ou minúsculas (“x” é diferente de “X”).

Para criar vetores de caracter, sempre colocamos os elementos entre aspas. Por exemplo:

```
frutas <- c("banana", "maçã", "uva", "manga")
frutas
```

```
## [1] "banana" "maçã" "uva" "manga"
```

Podemos fazer diversas operações com os vetores, como cálculos matemáticos:

```
peso.hoje <- c(55,60,70,75)
peso.ontem <- c(50,65,80,75)
peso.hoje - peso.ontem
```

```
## [1] 5 -5 -10 0
```

Podemos também ordenar os vetores com a função “sort”.

```
sort(frutas)
```

```
## [1] "banana" "maçã" "manga" "uva"
```

Dataframes

Os **dataframes** são objetos em forma de tabela que podem armazenar informações de diferentes tipos. Para criar dataframes a partir de vetores, utilizamos a função “data.frame”.

```
frutas <- c("banana", "maçã", "uva", "manga")
quantidade <- c(100,150,30,50)
vendas <- c(30,100,15,50)
importada <- c(FALSE,TRUE,TRUE,FALSE)

quitanda <- data.frame(frutas,quantidade,vendas,importada)
quitanda
```

```
##   frutas quantidade vendas importada
## 1 banana         100     30      FALSE
## 2 maçã          150    100       TRUE
## 3 uva           30     15       TRUE
## 4 manga          50     50      FALSE
```

Acessando os elementos de vetores e dataframes

Para acessar algum valor específico de um vetor utilizamos colchetes. Por exemplo, qual é a segunda fruta de nosso vetor de frutas?

```
frutas[2]
```

```
## [1] "maçã"
```

Para dataframes a lógica é semelhante. Porém, como agora temos linhas e colunas, precisamos indicar esses dois elementos dentro dos colchetes. Utilizamos uma vírgula dentro dos colchetes para indicar a linha e a coluna. O primeiro elemento indica a linha e o segundo a coluna.

```
quitanda[2,1]
```

```
## [1] "maçã"
```

Podemos também selecionar uma coluna inteira de um dataframe. Isso pode ser feito de duas formas diferentes:

```
quitanda[,1]
```

```
## [1] "banana" "maçã"   "uva"    "manga"
```

ou

```
quitanda$frutas
```

```
## [1] "banana" "maçã"   "uva"    "manga"
```

Workspace e diretório de trabalho

Quando trabalhamos em um projeto no R, além de salvar os nossos códigos em um script, é recomendável que nós criemos uma pasta onde iremos salvar tudo aquilo relacionado ao projeto, como os bancos de dados utilizados na análise e os gráficos gerados pelo R. Essa pasta é o nosso **diretório de trabalho**.

Então, antes de começarmos as nossas análises no R, o ideal é indicarmos qual é o nosso diretório de trabalho. Fazemos isso por meio da função “setwd”.

```
setwd("C:\\Users\\Mauricio\\Desktop\\cursoR\\")
```

Acima está o meu diretório de trabalho. É uma pasta localizada na minha Área de Trabalho com o nome “cursoR”. Não esqueça de utilizar a barra dupla (“\”) no endereço. Ou, se preferir, você pode utilizar uma única barra invertida (“/”):

```
setwd("C:/Users/Mauricio/Desktop/cursoR/")
```

Além do nosso script com todos os nossos códigos, também podemos salvar a imagem do R com todos os objetos que foram criados durante a análise. Isso é útil quando trabalhamos com muitas informações que demoram para ser processadas. Ao salvar a imagem do R, não precisamos rodar todo o script novamente para ter os nossos objetos. Para isso utilizamos a função “save.image”.

```
save.image("aula1.RData")
```

Importando e exportando dados

De modo geral, nós trabalhamos com bancos de dados que já estão prontos e que foram produzidos em outro software, como o **Excel**. Não criamos nossos dataframes no R, mas carregamos as tabelas no R.

É possível carregar bancos de dados de diferentes formatos no R. Eles podem ser, por exemplo, arquivos “.csv”, “.txt”, “.dta” e “.xlsx”, apenas para citar os formatos mais comuns.

Para carregar um arquivo “.csv”, utilizamos a função “read.csv”.

```
tab <- read.csv("candidatos.csv")
```

A função “read.csv” possui muitos argumentos. Veja o help dessa função para se familiarizar como ela. Um argumento importante dela é o “fileEncoding”. Neste argumento indicamos o tipo de codificação binária utilizada no arquivo. Isto é importante sobretudo quando o arquivo possui entradas com caracteres especiais, como acentos, cedilha e etc.

```
tab <- read.csv("candidatos.csv", fileEncoding = "latin1")
```

No exemplo acima, estamos carregando o arquivo “candidatos.csv” que está salvo em meu diretório de trabalho. Neste arquivo nós temos os dados dos candidatos a prefeito nas eleições de 2004 e 2008 em municípios com menos de 200 mil eleitores registrados e que terminaram na primeira ou segunda colocação.

Para olhar as bases de dados no R existe alguns comandos úteis:

- `head(tab)`: para ver as 6 primeiras linhas
- `tail(tab)`: para ver as 6 últimas linhas
- `View(tab)`: para ver toda a tabela

Vamos explorar as 6 primeiras linhas do nosso banco de dados:

```
head(tab)
```

```
##      SGL_UF      MUNICIPIO ELEICAO      NOME      DT_NASC
## 1      AC      ACRELANDIA      2004 PAULO CESAR FERREIRA DE ARAUJO 1957-04-08
## 2      AC      ACRELANDIA      2004 SEBASTIAO BOCALOM RODRIGUES 1953-05-18
## 3      AC      ACRELANDIA      2008 CARLOS CESAR NUNES DE ARAUJO 1984-07-17
## 4      AC      ACRELANDIA      2008 VILSEU FERREIRA DA SILVA 1960-04-18
## 5      AC ASSIS BRASIL      2004 ANTONIO BARBOSA DE SOUSA 1956-08-28
## 6      AC ASSIS BRASIL      2004 MANOEL BATISTA DE ARAUJO 1968-03-03
##      SEXO SGL_PARTIDO      VV1T eleito
## 1 Masculino      PTB 38.69998      0
## 2 Masculino      PSDB 40.82284      1
## 3 MASCULINO      PSB 47.55000      0
## 4 MASCULINO      PP 48.57000      1
## 5 Masculino      PPS 44.97372      0
## 6 Masculino      PT 55.02628      1
```

Também podemos exportar os dados que processamos no R. Basta utilizar a função “`write.csv`”.

```
write.csv(tab, "candidatos2.csv")
```

Essa função possui dois argumentos principais. O primeiro refere-se ao dataframe que queremos exportar. No segundo argumento, indicamos o nome que queremos dar ao arquivo.

Pacotes

Ao instalarmos o R em nosso computador, ele vem apenas com um conjunto básico de funções (base). Porém, uma das principais vantagens que o R possui em relação a outros softwares estatísticos é que ele está em constante desenvolvimento. Novas funções são criadas diariamente por muitas pessoas. E essas novas funções podem ser utilizadas por qualquer um. Para isso, precisamos instalar os **pacotes** com essas novas funções.

Para instalar um pacote no R, utilizamos a função “`install.packages`”.

```
install.packages("readxl")
```

Ao executar a função irá aparecer uma caixa em que devemos selecionar o repositório de onde iremos instalar o pacote. Selecione um (“Brazil (SP 1)”, por exemplo) e a instalação é automática.

Após a instalação, precisamos carregar o pacote com a função “`library`”. Precisamos instalar o pacote uma única vez, mas sempre que fecharmos o R precisaremos carregar novamente o

pacote para poder utilizar as suas funções.

```
library(readxl)
```

```
## Warning: package 'readxl' was built under R version 4.3.3
```

O pacote “readxl” traz diversas funções que nos ajudam a carregar um arquivo em Excel no R. Uma delas é a função “read_excel”.

```
tab <- read_excel("candidatos.xlsx")
head(tab)
```

```
## # A tibble: 6 x 9
##   SGL_UF MUNICIPIO      ELEICAO NOME      DT_NASC SEXO  SGL_PARTIDO  VV1T eleito
##   <chr>  <chr>          <dbl> <chr>    <chr>  <chr> <chr>      <dbl>  <dbl>
## 1 AC     ACRELANDIA      2004 PAULO CESA~ 1957-0~ Masc~ PTB          38.7    0
## 2 AC     ACRELANDIA      2004 SEBASTIAO ~ 1953-0~ Masc~ PSDB         40.8    1
## 3 AC     ACRELANDIA      2008 CARLOS CES~ 1984-0~ MASC~ PSB          47.6    0
## 4 AC     ACRELANDIA      2008 VILSEU FER~ 1960-0~ MASC~ PP           48.6    1
## 5 AC     ASSIS BRASIL    2004 ANTONIO BA~ 1956-0~ Masc~ PPS          45.0    0
## 6 AC     ASSIS BRASIL    2004 MANOEL BAT~ 1968-0~ Masc~ PT           55.0    1
```

Note que o banco de dados carregado por meio da função “read_excel” não está no formato dataframe, mas sim em **tibble**, que nada mais é do que uma versão moderna do dataframe.

Limpendo o banco de dados

O primeiro passo, antes de fazer qualquer análise estatística dos dados, é checar a consistência dos dados, isto é, verificar a existência de possíveis erros e corrigí-los. Qualquer um que já trabalhou com um banco de dados antes sabe que erros são comuns (mesmo em dados administrativos fornecidos pelo governo) e que boa parte do trabalho na pesquisa empírica está no processamento dos dados.

Há várias formas de checar a consistência dos dados e o R fornece várias funções úteis para isso.

Máximos e mínimos

Uma primeira coisa que podemos fazer é verificar os valores máximos e mínimos de variáveis numéricas em busca de valores que não estão dentro dos limites que deveriam estar. Por exemplo, a porcentagem de votos que um candidato recebeu em uma eleição deve estar sempre entre 0% e 100%. Vamos utilizar as funções “max” e “min” para fazer esses cálculos.

```
max(tab$VV1T)
```

```
## [1] 99.98
```

```
min(tab$VV1T)
```

```
## [1] 0.02
```

Tabulações

Podemos também fazer algumas tabulações para verificarmos se as variáveis de nosso banco de dados possuem algum valor errado. Para isso existe a função “table”.

```
table(tab$SGL_UF)
```

```
##
##   AC   AL   AM   AP   BA   CE   ES   GO   MA   MG   MS   MT   PA   PB   PE   PI
##   86  392  242   62 1642  718  288  944  856 3292  294  534  542  864  702  844
##   PR   RJ   RN   RO   RR   RS   SC   SE   SP   TO
## 1482  314  642  200   56 1852 1122  290 2398  540
```

Acima temos a tabulação da variável “SGL_UF”, a sigla da UF em que ocorreu a eleição para prefeito. Podemos observar que há 26 estados, sendo MG o com o maior número de observações (3292) e RR com o menor (56). Vemos também que não há o DF, porque não há eleições para prefeito. Finalmente, vemos que o número de observações em cada estado é sempre um número par. Isso faz sentido porque nós temos apenas os dois primeiros colocados nas eleições. Todos esses resultados são indícios que esta variável não possui erros.

Podemos fazer também uma tabulação entre duas variáveis, como as variáveis “SGL_UF” e “eleito”

```
table(tab$SGL_UF, tab$eleito)
```

```
##
##           0     1
##   AC     43    43
##   AL    196   196
##   AM    121   121
##   AP     31    31
##   BA    821   821
##   CE    359   359
##   ES    144   144
##   GO    472   472
##   MA    428   428
##   MG   1646  1646
##   MS    147   147
##   MT    267   267
##   PA    271   271
##   PB    432   432
##   PE    351   351
##   PI    422   422
##   PR    741   741
##   RJ    157   157
```

```
## RN 321 321
## RO 100 100
## RR 28 28
## RS 926 926
## SC 561 561
## SE 145 145
## SP 1199 1199
## TO 270 270
```

Vemos que o número de eleitos (1) e não eleitos (0) por UF é igual. Isso faz sentido, pois a base de dados contém apenas os 2 primeiros colocados nas eleições para prefeito.

Vejam agora a variável “SEXO”.

```
table(tab$SEXO)
```

```
##
##      Feminino      FEMININO      Masculino      MASCULINO Não informado
##           925           1153           9746           9315             59
```

Aqui observamos o primeiro erro. As categorias “masculino” e “feminino” estão escritas de duas formas diferentes, uma com a primeira letra maiúscula e a outra com todas as letras maiúsculas. Precisamos corrigir esse problema, já que o R é *case sensitive*.

Função ifelse

A função “ifelse” faz o seguinte teste: se o teste X for verdadeiro, então retorne o valor A, caso contrário retorne o valor B. Vamos dar um exemplo para deixar isso mais claro.

```
x <- c(1:5)
ifelse(x > 3, "Maior", "Menor ou igual")
```

```
## [1] "Menor ou igual" "Menor ou igual" "Menor ou igual" "Maior"
## [5] "Maior"
```

No código acima temos um vetor numérico que vai de 1 a 5 (x). Então testamos a proposição “x > 3”. Como x é um vetor, o teste é aplicado para cada um dos cinco números que compõem o vetor x. Quando x é maior do que 3 (x=4 ou x=5), o R retorna a palavra “Maior”, mas quando x é menor ou igual a 3 (x=1, x=2 ou x=3), ele retorna a palavra “Menor ou igual”.

Os operadores lógicos mais utilizados são estes:

- “==”: Igual
- “!=”: Diferente
- “>”: Maior
- “<”: Menor
- “&”: E
- “|”: Ou

Agora vamos utilizar essa função para corrigir a variável “SEXO”. Podemos fazer isso corrigindo uma categoria de cada vez. Primeiro a categoria “FEMININO”.

```
tab$SEXO <- ifelse(tab$SEXO == "FEMININO", "Feminino", tab$SEXO)
table(tab$SEXO)
```

```
##
##      Feminino      Masculino      MASCULINO Não informado
##           2078           9746           9315           59
```

Agora a categoria “MASCULINO”.

```
tab$SEXO <- ifelse(tab$SEXO == "MASCULINO", "Masculino", tab$SEXO)
table(tab$SEXO)
```

```
##
##      Feminino      Masculino Não informado
##           2078           19061           59
```

Outra forma mais elegante de se fazer isso seria utilizando dois ifelse’s de uma única vez.

```
tab <- read_excel("candidatos.xlsx") #vamos carregar a tabela novamente

tab$SEXO <- ifelse(tab$SEXO == "MASCULINO", "Masculino",
                  ifelse(tab$SEXO == "FEMININO", "Feminino", tab$SEXO))
table(tab$SEXO)
```

```
##
##      Feminino      Masculino Não informado
##           2078           19061           59
```

Aqui aplicamos uma estrutura um pouco mais complexa, pois concatemos um ifelse dentro de outro ifelse.

Vejamos quantas mulheres foram eleitas para o cargo de prefeito nas eleições de 2004 e 2008:

```
table(tab$SEXO, tab$eleito)
```

```
##
##              0      1
## Feminino    1202  876
## Masculino   9368 9693
## Não informado  29   30
```

Datas (as.Date)

Quando carregamos um banco de dados no R, em geral, variáveis com datas (como a data de nascimento de um candidato) são carregadas como uma variável de caracteres. Se quisermos fazer cálculos com datas, precisamos transformá-las em uma variável de tipo data. Para isso temos a função “as.Date”. Essa função possui dois argumentos principais. O primeiro contém o

vetor com as datas (x) e o segundo o formato da data (format). Visite a página de ajuda da função para mais detalhes.

```
tab$DT_NASC <- as.Date(tab$DT_NASC, format = "%Y-%m-%d")
```

O formato “%Y-%m-%d” indica que primeiro temos o ano com 4 dígitos, depois, separado por um traço, o mês e, finalmente, o dia.

Com isso podemos calcular, por exemplo, a idade do candidato no dia da eleição. As eleições de 2004 ocorreram em 03/10/2004 e as de 2008 em 05/10/2008.

```
dt2004 <- as.Date("03/10/2004", format = "%d/%m/%Y")
dt2008 <- as.Date("05/10/2008", format = "%d/%m/%Y")
```

```
tab$idade <- ifelse(tab$ELEICAO == 2004,
                    (dt2004 - tab$DT_NASC)/365.25,
                    (dt2008 - tab$DT_NASC)/365.25)
```

```
max(tab$idade, na.rm = T)
```

```
## [1] 90.55989
```

```
min(tab$idade, na.rm = T)
```

```
## [1] 20.78029
```

Neste exemplo, criamos, em primeiro lugar, as datas das eleições de 2004 (dt2004) e 2008 (dt2008). Em seguida, utilizamos a estrutura do ifelse para fazer o teste lógico. Se “ELEICAO” for igual a 2004, então subtraímos “dt2004” da data de nascimento do candidato e dividimos por 365.25. Caso contrário, a subtração será feita com “dt2008”. A divisão por 365.25 ocorre porque queremos calcular a idade em anos, não em dias (e o .25 é por conta dos anos bissextos).

Exercícios

1. Calcule a raiz quadrada de 36.
2. Calcule a raiz cúbica de 27.
3. Crie o vetor `x <- c(1,2,3,"a")`. O que ocorre com os números do vetor `x`?
4. No banco de dados “sen_x_votacao.txt” temos as votações nominais que ocorreram no Senado Federal entre 20 de março e 30 de junho de 2020. Carregue esse banco de dados no R utilizando a função “read.table”. (**Dica:** veja os argumentos “sep” e “header” dessa função)
5. A partir do banco de dados de candidatos (“tab”), crie uma variável nova com o nome “regiao” na qual você indicará a região do país (Norte, Nordeste, Centro-Oeste, Sul e Sudeste) a partir da variável “SGL_UF”. (**Dica:** utilize o operador “%in%” para fazer o teste lógico ao invés de “==”)

6. Encontre um pacote do R para exportar dados em formato “.xlsx” (Excel). Instale e carregue o pacote e exporte o objeto “tab” com os candidatos a prefeito para o seu diretório de trabalho.
7. Crie o vetor `x <- “Palmeiras é o maior campeão do Brasil”`. Encontre uma função que conte o número de caracteres nesse vetor, incluindo os espaços.
8. A partir do mesmo vetor criado no exercício anterior, encontre uma função para calcular o número de palavras no vetor.
9. Crie o dataframe abaixo:

```
Item <- c("Caderno", "Lápis", "Caneta", "Borracha", "Apontador", "Régua")
Quantidade <- c(10, 20, 20, 15, 5, 2)
PrecoUnitCompra <- c(15, 0.5, 1.5, 0.75, 2.5, 5)
PrecoUnitVenda <- c(20, 2, 2.5, 1.5, 7, 12)
tab <- data.frame(Item, Quantidade, PrecoUnitCompra, PrecoUnitVenda)
```

Aqui temos os dados de uma papelaria. Temos o item, a quantidade do produto comercializada, o preço unitário de compra de cada item e o preço unitário de venda de cada item. Calcule o lucro total do dono da papelaria.

10. Com base na tabela do exercício anterior, suponha que o dono da papelaria faça uma promoção e dê um desconto de 10% no lápis e na caneta. Qual será o seu lucro agora?